BombLab 实验报告

周益轩 25303090066

2026 年 10 月 26 日

1 实验目的与方法

本实验通过分析 x86-64 程序的反汇编，恢复各关卡从 64 位 `challenge` 到口令的计算关系，并在 `defuser.c` 中实现对应求解过程。分析遵循 System V AMD64 调用约定：前两个整型参数分别通过 `rdi` 和 `rsi` 传递，返回值通过 `rax` 传递。本报告不逐行翻译完整汇编，而是记录决定输入拆分、函数参数和返回值的关键指令。

2 Phase 1：计算关系与求解

2.1 关键推导

`p1_compute` 对 `challenge` 进行三种 32 位截取。与求解代码中的变量名保持一致，记为

$$\begin{aligned} \texttt{x_b32} &= \texttt{challenge} \,\&\, \texttt{0xffffffff}, \\ \texttt{x_f32} &= (\texttt{challenge} \gg 32) \,\&\, \texttt{0xffffffff}, \\ \texttt{x_r16} &= (\texttt{challenge} \gg 16) \,\&\, \texttt{0xffffffff}. \end{aligned}$$

第一次调用中，`p1_step2` 的两个参数分别为 `x_f32` 和 `x_b32`。关键指令为

```
lea  (%rdi,%rdi,4),%eax
add  %esi,%eax
xor  $0xcac5,%eax
```

因此得到

$$\texttt{S2} = (5\texttt{x_f32} + \texttt{x_b32}) \oplus \texttt{0xcac5}. \tag{1}$$

随后 `p1_step0` 以 `x_b32` 和 `x_r16` 为参数：

```
lea  0x7032(%rsi),%eax
xor  %edi,%eax
```

对应的中间结果为

$$\texttt{S0} = (\texttt{x_r16} + \texttt{0x7032}) \oplus \texttt{x_b32}. \tag{2}$$

最后，`p1_step1` 将 `S0` 与 `S2` 合并：

```
xor  $0x5d86,%esi
lea  (%rsi,%rdi,4),%eax
```

因而最终口令为

$$\texttt{S1} = (\texttt{S2} \oplus \texttt{0x5d86}) + 4\texttt{S0} \pmod{2^{32}}. \tag{3}$$

2.2 完整代码

```
int solve_phase_1(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 1;

    (void)challenge;

    uint32_t x_b32 = challenge & 0xffffffff;
    uint32_t x_f32 = (challenge >> 32) & 0xffffffff;
    uint32_t x_r16 = challenge >> 16;

    uint32_t S2 = (x_f32 * 5 + x_b32) ^ 0xcac5;
    uint32_t S0 = (x_r16 + 0x7032) ^ x_b32;
    uint32_t S1 = (S2 ^ 0x5d86) + 4*S0;
    values[0] = S1;

    return write_answer(out, cap, values, count);
}
```

2.3 运行结果



图 1　Phase 1 公开列表外种子运行结果

```
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
```

图 2　Phase 1 多种子测试结果

3 Phase 2：数组解析与递推关系

3.1 关键推导

本关最终需要给出六个无符号整数,因此求解代码使用 `values[0]` 至 `values[5]` 保存结果，并将 `count` 设为 6。函数入口先将第二个参数 `challenge` 保存到 `r13`，随后用以下关键指令检查第一个整数：

```
mov  %rsi,%r13
mov  %r13,%rax
mov  -0x40(%rbp),%esi
shr  $0x2f,%rax
and  $0x1f,%eax
lea  (%rax,%rax,2),%eax
cmp  %eax,%esi
jne  402115
```

其中 `-0x40(%rbp)` 是输入数组首元素的位置；右移 `0x2f`=47 位后，`and` 以 `0x1f` 截取 5 位，`lea (%rax,%rax,2)` 则计算该数的 3 倍。最后的 `cmp` 表明输入的第一项必须等于这一结果。对应到代码变量，有

$$\texttt{values[0]} = 3\,((\texttt{challenge} \gg 47)\,\&\,\texttt{0x1f})\,. \tag{4}$$

首项检查通过后，`ecx` 被置为 6，`edx` 被置为 1，分别对应代码中的 `bias` 和 `i`。剩余五项由下面的循环递推得到：

```
mov  $0x6,%ecx
mov  $0x1,%edx
mov  %r13,%rax
lea  (%rsi,%rsi,4),%esi
shr  %cl,%rax
and  $0x3f,%eax
add  %edx,%eax
xor  %esi,%eax
mov  -0x40(%rbp,%rdx,4),%esi
cmp  %eax,%esi
jne  402115
add  $0x1,%rdx
add  $0x4,%ecx
cmp  $0x6,%rdx
jne  402178
```

这里 `lea (%rsi,%rsi,4)` 将上一项乘以 5；`shr %cl` 说明右移量取自 `ecx` 的低 8 位，也就是 `bias`。随后程序用 `0x3f` 截取 6 位，加上当前下标 `i`，再与上一项的 5 倍进行按位异或。由此，循环中的计算关系可直接写为

$$\texttt{values[i]} = (((\texttt{challenge} \gg \texttt{bias})\,\&\,\texttt{0x3f}) + \texttt{i}) \oplus (5\,\texttt{values[i-1]})\,. \tag{5}$$

每轮结束后执行 `i += 1` 和 `bias += 4`。因而第 i 项使用的移位量为 $6+4(i-1)$，循环条件 `i < 6` 恰好生成 `values[1]` 至 `values[5]`。所有中间量均采用 `uint32_t`，使乘法、加法和异或自然保持与 32 位寄存器相同的模 2^{32} 运算语义。

3.2 完整代码

```
int solve_phase_2(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 6;

    values[0] = 3 * ((challenge >> 47) & 0x1f);

    uint32_t i = 1;
    uint32_t bias = 6;

    while (i < 6) {
        values[i] = ((((challenge >> bias) & 0x3f) + i)
                     ^ (values[i - 1] * 5));
        i += 1;
        bias += 4;
    }

    return write_answer(out, cap, values, count);
}
```

3.3 运行结果



图 3　Phase 2 公开列表外种子运行结果

```
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
```

图 4　Phase 2 多种子测试结果

4 Phase 3：跳转表与分支计算

4.1 关键推导

`phase_03` 先调用 `valid_u32_fields(input, 2)`，随后使用 `sscanf` 读取两个无符号 32 位整数，并检查返回值是否为 2。因此本关口令可记为 v_0, v_1。程序将入口处保存在 `r12` 中的 `challenge` 变换为两个派生量。第一项检查的关键指令为

```
mov  %r12d,%edi
and  $0x7,%edi
cmp  %edi,-0x18(%rbp)
jne  failure
```

所以代码中的 a 以及第一个输出为

$$\texttt{a} = \texttt{challenge} \,\&\, \texttt{0x07}, \qquad \texttt{values[0]} = \texttt{a}. \tag{6}$$

第二个派生量由以下指令得到：

```
mov  %r12,%rax
mov  %r12,%rsi
shr  $0x3,%rax
shr  $0x20,%rsi
xor  %eax,%esi
```

两条 shr 在 64 位寄存器上执行，而最后的 xor 使用两个 32 位子寄存器。因此

$$\texttt{b} = (\texttt{challenge} \gg \texttt{0x20}) \oplus (\texttt{challenge} \gg 3). \tag{7}$$

间接跳转表。 p3_dispatch 先检查 a 是否处于 0 至 7，再将其作为跳转表下标：

```
mov     %edi,%eax
mov     %esi,%edi
cmp     $0x7,%eax
ja      default
notrack jmp *0x405820(,%rax,8)
```

ja 按无符号数比较，故只有 $0 \le a \le 7$ 能进入跳转表。间接跳转中的地址表达式表示每个表项占 8 字节，目标地址为表中第 a 项保存的地址。使用 GDB 命令 x/8a 0x405820 读取表项，并继续追踪各中转地址，可得到

a	0	1	2	3
目标函数	case_1	case_6	case_7	case_4
a	4	5	6	7
目标函数	case_0	case_5	case_2	case_3

由此可见，输入下标与辅助函数名称中的编号并不相同，不能直接假定 $a = i$ 时调用 p3_case_i。

八个分支的计算关系。 进入目标函数前，mov %esi,%edi 将 b 作为其第一个参数。各辅助函数中决定结果的关键指令如下：

```
p3_case_1: lea 0xc0fe(%rdi,%rdi,2),%eax
p3_case_6: lea (%rdi,%rdi,1),%eax; xor $0x9ed3,%eax
```

```
p3_case_7: lea 0x2b29(%rdi,%rdi,2),%eax
p3_case_4: mov %edi,%eax; shr $1,%edi
           xor $0x62e8,%eax; add %edi,%eax
p3_case_0: lea 0x0(,%rdi,4),%eax; xor $0x49e5,%eax
p3_case_5: lea 0x3a813(%rdi,%rdi,4),%eax
p3_case_2: lea 0x3d202(%rdi,%rdi,4),%eax
p3_case_3: mov %edi,%eax; shr $1,%edi
           xor $0xef6d,%eax; add %edi,%eax
```

其中 `lea offset(%rdi,%rdi,k)` 表示 $(1+k)\texttt{b}+\textit{offset}$。两个包含 `shr` 的分支则分别计算 `b >> 1` 和 `b` 异或常量，再将两者相加。结合跳转表映射，第二个输出 `values[1]` 应满足

$$\texttt{values[1]} = \begin{cases} 3b + \texttt{0xc0fe}, & a = 0, \\ (2b) \oplus \texttt{0x9ed3}, & a = 1, \\ 3b + \texttt{0x2b29}, & a = 2, \\ (b \gg 1) + (b \oplus \texttt{0x62e8}), & a = 3, \\ (4b) \oplus \texttt{0x49e5}, & a = 4, \\ 5b + \texttt{0x3a813}, & a = 5, \\ 5b + \texttt{0x3d202}, & a = 6, \\ (b \gg 1) + (b \oplus \texttt{0xef6d}), & a = 7. \end{cases} \pmod{2^{32}} \tag{8}$$

例如，$a = 3$ 实际进入 `p3_case_4`，其中异或常量为 `0x62e8`；$a = 7$ 实际进入 `p3_case_3`，其中异或常量为 `0xef6d`。这两个分支的结构相同但常量不同，分析时需要同时核对跳转映射和目标函数，避免交换常量。

4.2 完整代码

```c
int solve_phase_3(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 2;
    uint32_t a = challenge & 0x07;
    uint32_t b = (challenge >> 0x20) ^ (challenge >> 3);
    values[0] = a;
    switch (a) {
        case 0: values[1] = b * 3 + 0xc0fe; break;
        case 1: values[1] = (b * 2) ^ 0x9ed3; break;
        case 2: values[1] = b * 3 + 0x2b29; break;
        case 3: values[1] = (b >> 1) + (b ^ 0x62e8); break;
        case 4: values[1] = (b * 4) ^ 0x49e5; break;
        case 5: values[1] = b * 5 + 0x3a813; break;
        case 6: values[1] = b * 5 + 0x3d202; break;
        case 7: values[1] = (b >> 1) + (b ^ 0xef6d); break;
    }
    return write_answer(out, cap, values, count);
}
```

4.3 运行结果



图 5　Phase 3 公开列表外种子运行结果

```
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
```

图 6　Phase 3 多种子测试结果

5 Phase 4：递归调用与返回过程

5.1 关键推导

`phase_04` 先调用 `valid_u32_fields(input, 1)`，随后用 `sscanf` 读取一个无符号 32 位整数，并检查返回值是否为 1。因此本关只需输出一个整数。函数入口处的 `r12` 保存第二个参数 `challenge`，调用递归函数前的关键指令为

```
mov  %r12,%rcx
mov  %r12,%rax
mov  %r12d,%edi
xor  %esi,%esi
shr  $0x20,%rax
shr  $0x7,%rcx
and  $0x7f,%edi
mov  $0x7f,%edx
xor  %eax,%ecx
call p4_walk
```

由上述调用点的寄存器赋值可得 `p4_walk` 的初始参数：

$$
\begin{aligned}
a &= (\texttt{uint32_t})\texttt{challenge} \,\&\, \texttt{0x7f},\\
b &= 0,\\
c &= \texttt{0x7f},\\
d &= (\texttt{uint32_t})\left((\texttt{challenge} \gg 32) \oplus (\texttt{challenge} \gg 7)\right).
\end{aligned}
$$

中点计算与终止条件。 `p4_walk` 入口处先将原始的 a 从 `edi` 复制到 `eax`，随后复用 `edi` 计算区间中点：

```
mov  %edi,%eax
mov  %edx,%edi
```

```
sub  %esi,%edi
shr  $1,%edi
add  %esi,%edi
cmp  %eax,%edi
je   base_case
```

AT&T 语法的赋值方向为“源到目标”，所以第一条指令是 `eax = edi`。执行到比较指令时，`eax` 保存原始参数 a，而 `edi` 中为

$$m = b + ((c - b) \gg 1). \tag{9}$$

由于寄存器和参数均按无符号 32 位数处理，`shr` 对应逻辑右移。当 $a = m$ 时进入递归终止分支：

```
lea  0x97(%rdi),%eax
xor  %ecx,%eax
ret
```

因此基础情况为

$$W(a, b, c, d) = (m + \texttt{0x97}) \oplus d, \qquad a = m. \tag{10}$$

两个递归分支与返回值合成。 若 $a \neq m$，函数先用 `lea (%rdi,%rcx,1),%ebx` 保存 $m + d$，随后执行

```
cmp  %edi,%eax
jb   lower_branch
```

`cmp %edi,%eax` 设置 $a - m$ 对应的条件码，`jb` 是无符号“低于”跳转，所以该分支条件为 $a < m$。在该分支中，程序令 $c = m - 1$ 后递归调用；否则因相等情况已经排除，必有 $a > m$，程序令 $b = m + 1$ 后递归。两条路径的关键指令为

```
a < mid: lea -0x1(%rdi),%edx; call p4_walk
         lea (%rax,%rax,4),%eax; xor %ebx,%eax
         xor $0x9,%eax
a > mid: lea 0x1(%rdi),%esi; call p4_walk
         lea (%rax,%rax,4),%eax; xor %ebx,%eax
         xor $0x1f,%eax
```

其中 `lea (%rax,%rax,4)` 计算递归结果的 5 倍，`ebx` 保存此前得到的 `mid + d`。因此完整递推关系为

$$W(a, b, c, d) = \begin{cases} (m + \texttt{0x97}) \oplus d, & a = m, \\ 5W(a, b, m - 1, d) \oplus (m + d) \oplus \texttt{0x9}, & a < m, \\ 5W(a, m + 1, c, d) \oplus (m + d) \oplus \texttt{0x1f}, & a > m, \end{cases} \pmod{2^{32}}. \tag{11}$$

递归分支使用 `rbx` 跨调用保存 $m+d$。修改该被调用者保存寄存器前，函数先将其压栈，并在返回前通过 `mov -0x8(%rbp),%rbx` 恢复。随后，`leave` 销毁当前栈帧，后续的 `ret` 才真正返回调用者。

5.2 完整代码

```
static uint32_t p4_walk(uint32_t a, uint32_t b,
                        uint32_t c, uint32_t d) {
    uint32_t mid = b + ((c - b) >> 1);
    if (a == mid)
        return (mid + 0x97u) ^ d;
    if (a < mid)
        return (5u * p4_walk(a, b, mid - 1u, d))
               ^ (mid + d) ^ 0x09u;
    return (5u * p4_walk(a, mid + 1u, c, d))
           ^ (mid + d) ^ 0x1fu;
}

int solve_phase_4(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 1;
    uint32_t a = challenge & 0x7f;
    uint32_t b = 0;
    uint32_t c = 0x7f;
    uint32_t d = (challenge >> 0x20) ^ (challenge >> 0x7);
    values[0] = p4_walk(a, b, c, d);
    return write_answer(out, cap, values, count);
}
```

5.3 运行结果



图 7　Phase 4 公开列表外种子运行结果

```
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
```

图 8　Phase 4 多种子测试结果

6 Phase 5：链表选择、遍历与数据重建

6.1 关键推导

`phase_05` 首先调用 `valid_u32_fields(input, 2)`，说明本关要求输入两个无符号 32 位整数。随后，`sscanf` 将它们分别写入 `-0x1c(%rbp)` 和 `-0x18(%rbp)`，并检查返回值是否为 2。调用 `p5_traverse` 时，`rdi` 中是 64 位 `challenge`，`rsi` 则指向局部变量 `-0x14(%rbp)`：

```
lea  -0x14(%rbp),%rsi
```

```
mov  %r12,%rdi
call p5_traverse
mov  %eax,%edx
cmp  %edx,-0x1c(%rbp)
jne  failure
mov  -0x14(%rbp),%eax
cmp  %eax,-0x18(%rbp)
sete %al
```

因此，第一个口令整数是 `p5_traverse` 在 `eax` 中返回的累计值；第二个口令整数是该函数经第二个指针参数写回的值。

链表头的选择。 `p5_traverse` 使用以下指令选择链表头：

```
mov  %rdi,%rax
and  $0x7,%eax
mov  0x405b60(,%rax,8),%rdx
```

`and $0x7,%eax` 将索引限制在 0 至 7；随后以比例因子 8 访问 `0x405b60` 开始的内存。由于 64 位指针占 8 字节，该寻址形式表明 `0x405b60` 是含有八个元素的链表头指针表。使用 GDB 命令 `x/8gx 0x405b60` 得到图 9 所示结果。

```
(gdb) x/8gx 0x405b60
0x405b60 <p5_heads>:    0x0000000000405be0      0x0000000000405c30
0x405b70 <p5_heads+16>: 0x0000000000405c40      0x0000000000405bd0
0x405b80 <p5_heads+32>: 0x0000000000405c00      0x0000000000405bc0
0x405b90 <p5_heads+48>: 0x0000000000405bf0      0x0000000000405bb0
```

图 9　使用 GDB 提取八个链表头指针

将地址 `0x405ba0` 处的第一个节点编号为 0，并注意到相邻节点相差 `0x10`=16 字节，可由

$$i = \frac{\texttt{address} - \texttt{0x405ba0}}{16} \tag{12}$$

把节点地址转换成数组下标。八个头指针 `0x405be0`、`0x405c30`、`0x405c40`、`0x405bd0`、`0x405c00`、`0x405bc0`、`0x405bf0`、`0x405bb0` 因而分别对应下标

$$H = [4, 9, 10, 3, 6, 2, 5, 1]. \tag{13}$$

初始节点下标可写为

$$j = H[\texttt{challenge} \,\&\, 7]. \tag{14}$$

节点结构与 GDB 数据提取。 对当前节点的访问集中在以下三条指令：

```
mov  (%rdx),%rdx
```

```
mov  0x8(%rdx),%esi
add  0xc(%rdx),%eax
```

第一条从节点偏移 0 处读取 8 字节的下一节点指针；后两条分别从偏移 8 和 12 处读取两个 32 位字段。因此节点结构为

```
struct Node {
    struct Node *next;  /* offset 0,  8 bytes */
    uint32_t x1;        /* offset 8,  4 bytes */
    uint32_t x2;        /* offset 12, 4 bytes */
};
```

图 10 使用三条 GDB 命令分别读取地址 `0x405bb0` 处节点的三个字段。结果表明该节点的 `next=0x405c20`、`x1=0x470`、`x2=0xe9be`，同时验证了上述字段偏移。

```
(gdb) x/gx 0x405bb0
0x405bb0 <p5_nodes+16>: 0x0000000000405c20
(gdb) x/wx 0x405bb8
0x405bb8 <p5_nodes+24>: 0x00000470
(gdb) x/wx 0x405bbc
0x405bbc <p5_nodes+28>: 0x0000e9be
```

图 10　分别读取一个节点的 `next`、`x1` 和 `x2` 字段

节点区从 `0x405ba0` 开始，共有 12 个 16 字节节点，故使用 `x/24gx 0x405ba0` 一次读取 24 个 8 字节量。原始内存如图 11 所示；进一步用 GDB 循环按字段格式化后得到图 12。对于每个节点，原始输出中的第一个 8 字节量是 `next`，第二个 8 字节量的低 32 位是 `x1`，高 32 位是 `x2`。

初始值、节点更新和遍历次数。　累计值的初始计算由以下指令给出：

```
mov  %rdi,%rax
shr  $0x20,%rdi
shr  $0x6,%rax
xor  %edi,%eax
xor  $0xd492,%eax
```

由于最终运算在 32 位的 `eax`、`edi` 上完成，可记

$$R_0 = (\texttt{uint32_t})(\texttt{challenge} \gg 6) \oplus (\texttt{uint32_t})(\texttt{challenge} \gg 32) \oplus \texttt{0xd492}. \quad (15)$$

处理当前节点时执行

```
add  0xc(%rdx),%eax
lea  (%rax,%rax,4),%eax
mov  %esi,(%r8)
xor  %esi,%eax
```

```
(gdb) x/24gx 0x405ba0
0x405ba0 <p5_nodes>:    0x0000000000000000      0x00007ef900000214
0x405bb0 <p5_nodes+16>: 0x0000000000405c20      0x0000e9be00000470
0x405bc0 <p5_nodes+32>: 0x0000000000405bf0      0x00002c2c00000621
0x405bd0 <p5_nodes+48>: 0x0000000000405c00      0x0000d13000000a4b
0x405be0 <p5_nodes+64>: 0x0000000000405c30      0x00007f0300000349
0x405bf0 <p5_nodes+80>: 0x0000000000405bb0      0x00001cfe00000021
0x405c00 <p5_nodes+96>: 0x0000000000405bc0      0x0000894f0000083f
0x405c10 <p5_nodes+112>:        0x0000000000405c50      0x00000fbf00000987
0x405c20 <p5_nodes+128>:        0x0000000000405c10      0x00001aca0000051a
0x405c30 <p5_nodes+144>:        0x0000000000405c40      0x00001a6100000111
0x405c40 <p5_nodes+160>:        0x0000000000405bd0      0x0000ae3700000b3d
0x405c50 <p5_nodes+176>:        0x0000000000405ba0      0x000056e00000070b
```

图 11　从 p5_nodes 起始地址读取全部节点的原始内存

```
node[0] addr=0x405ba0 next=(nil) x1=0x214 x2=0x7ef9
node[1] addr=0x405bb0 next=0x405c20 x1=0x470 x2=0xe9be
node[2] addr=0x405bc0 next=0x405bf0 x1=0x621 x2=0x2c2c
node[3] addr=0x405bd0 next=0x405c00 x1=0xa4b x2=0xd130
node[4] addr=0x405be0 next=0x405c30 x1=0x349 x2=0x7f03
node[5] addr=0x405bf0 next=0x405bb0 x1=0x21 x2=0x1cfe
node[6] addr=0x405c00 next=0x405bc0 x1=0x83f x2=0x894f
node[7] addr=0x405c10 next=0x405c50 x1=0x987 x2=0xfbf
node[8] addr=0x405c20 next=0x405c10 x1=0x51a x2=0x1aca
node[9] addr=0x405c30 next=0x405c40 x1=0x111 x2=0x1a61
node[10] addr=0x405c40 next=0x405bd0 x1=0xb3d x2=0xae37
node[11] addr=0x405c50 next=0x405ba0 x1=0x70b x2=0x56e0
```

图 12　按节点字段格式化后的链表数据

其中 `lea (%rax,%rax,4)` 计算 $5\texttt{rax}$，故对节点 N_j 的更新关系为

$$R \leftarrow 5(R + N_j.x2) \oplus N_j.x1 \pmod{2^{32}},$$
$$E \leftarrow N_j.x1.$$

E 即通过第二个参数写回的值。函数先处理头节点，再根据

$$s = (\texttt{challenge} \gg 3)\ \&\ 3 \tag{16}$$

沿 `next` 继续移动 s 次。循环末尾的 `cmp %ecx,%edi` 与 `jae` 按无符号数判断 $s \geq i$，所以当 $s = 1, 2, 3$ 时分别额外处理 1、2、3 个节点；当 $s = 0$ 时，`and` 设置零标志，紧随其后的 `je` 直接返回。

独立求解器中的链表重建。 绝对地址 `0x405b60` 只在原始 `bomb` 进程中有效，独立的 `defuser` 不能直接解引用该地址。因此实现时将 GDB 提取出的 12 个节点和 8 个头指针统一转换为数组下标，以等价重建链表结构。

6.2 完整代码

```
struct NodeData {
    int next;
    uint32_t x1;
    uint32_t x2;
};

static const struct NodeData nodes[] = {
    {-1, 0x0214, 0x7ef9}, { 8, 0x0470, 0xe9be},
    { 5, 0x0621, 0x2c2c}, { 6, 0x0a4b, 0xd130},
    { 9, 0x0349, 0x7f03}, { 1, 0x0021, 0x1cfe},
    { 2, 0x083f, 0x894f}, {11, 0x0987, 0x0fbf},
    { 7, 0x051a, 0x1aca}, {10, 0x0111, 0x1a61},
    { 3, 0x0b3d, 0xae37}, { 0, 0x070b, 0x56e0}
};
static const unsigned heads[] = {4, 9, 10, 3, 6, 2, 5, 1};

int solve_phase_5(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 2;
    unsigned index = heads[challenge & 0x7];
    uint32_t result = (challenge >> 0x6)
                      ^ (challenge >> 0x20) ^ 0xd492;
    uint32_t steps = (challenge >> 0x3) & 0x3;
    result = 5 * (result + nodes[index].x2) ^ nodes[index].x1;
    uint32_t last_x1 = nodes[index].x1;
    for (uint32_t i = 0; i < steps; ++i) {
        index = nodes[index].next;
        result = 5 * (result + nodes[index].x2) ^ nodes[index].x1;
        last_x1 = nodes[index].x1;
    }
    values[0] = result;
    values[1] = last_x1;
    return write_answer(out, cap, values, count);
}
```

6.3 运行结果



图 13　Phase 5 公开列表外种子运行结果

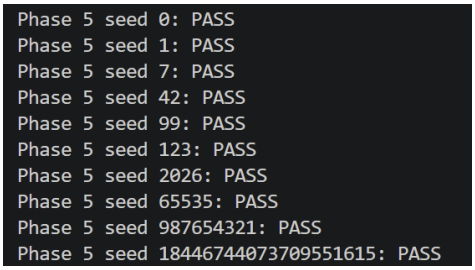


图 14　Phase 5 多种子测试结果

7 Phase 6：函数指针表与两级动态分派

7.1 关键推导

`phase_06` 首先调用 `valid_u32_fields(input, 1)`，因此本关只要求输入一个 32 位无符号整数。校验通过后，程序使用 `sscanf` 将该整数保存到局部变量，再把第二个参数中保存的 64 位 `challenge` 传给 `p6_dispatch`：

```
mov  %r12,%rdi
call p6_dispatch
cmp  %eax,-0x14(%rbp)
sete %al
```

因此，本关的目标是恢复 `p6_dispatch(challenge)` 的返回值，并将其作为唯一一个无符号十进制整数输出。比较发生在 32 位的 `eax` 和局部变量之间，所以后续计算均应按照模 2^{32} 的无符号运算处理。

初始值与两个函数下标。　`p6_dispatch` 先从 `challenge` 的不同位段构造辅助函数的输入。关键指令为

```
mov  %rdi,%rbx
shr  $0x6,%rdi
mov  %rbx,%rax
shr  $0x20,%rax
xor  %eax,%edi
```

由于最后的异或在 `edi` 和 `eax` 上进行，只有两个右移结果的低 32 位参与运算。记辅助函数的初始输入为 X_0，则

$$X_0 = (\texttt{uint32_t})(\texttt{challenge} \gg 6) \oplus (\texttt{uint32_t})(\texttt{challenge} \gg 32). \tag{17}$$

随后，原始 `challenge` 的低 6 位被拆成两个 3 位下标：

```
and  $0x38,%ebx
and  $0x7,%eax
call *0x405e00(,%rax,8)
```

第一次调用采用比例因子 8，因此 `rax` 直接表示函数表下标；第二次调用使用 `0x405e00(%rbx)`，而 `rbx` 已被限制为 $0, 8, \ldots, 56$，其字节偏移除以 8 后才是逻辑下标。于是

$$I_1 = \texttt{challenge} \,\&\, 7,$$
$$I_2 = (\texttt{challenge} \,\&\, \texttt{0x38}) \gg 3.$$

函数指针表的恢复。 间接调用的基址固定为 `0x405e00`，每个表项占 8 字节。使用 `x/8gx 0x405e00` 读取该区域，得到图 15 所示的八个函数地址。将地址交给 `info symbol` 或直接反汇编后，可以恢复表项与函数名的对应关系：

$$T = [f_1, f_1, f_4, f_3, f_5, f_1, f_2, f_0]. \tag{18}$$

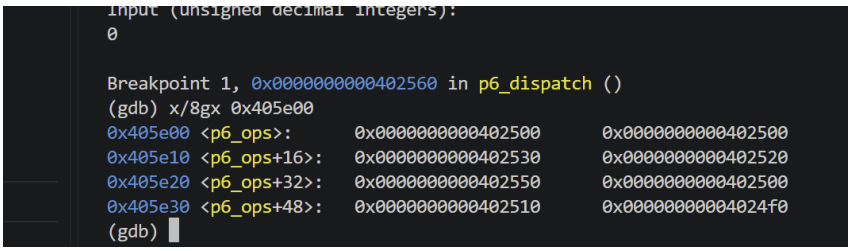


图 15　使用 GDB 读取 `p6_ops` 函数指针表

图中的八个 64 位数是函数入口地址，而不是普通整数。因此在独立求解器中应将该表声明为函数指针数组。例如，所有函数均接收并返回一个 `uint32_t` 时，可写为

```
typedef uint32_t (*op_func)(uint32_t);

static op_func ops[8] = {
    p6_op_1, p6_op_1, p6_op_4, p6_op_3,
    p6_op_5, p6_op_1, p6_op_2, p6_op_0
};
```

六个辅助函数的计算关系。 各函数均通过 `edi` 接收参数并通过 `eax` 返回结果。图 16 展示了六个函数的完整反汇编。分析时以每个 `ret` 作为函数边界，并将 `lea` 视为算术表达式，而不是内存读取。

记输入为 x，所有结果均按 32 位截断。六个函数可写为



```
(gdb) x/30i 0x402500
   0x402500 <p6_op_1>:  endbr64
   0x402504 <p6_op_1+4>:        lea    0x6fae(%rdi),%eax
   0x40250a <p6_op_1+10>:       shr    $1,%edi
   0x40250c <p6_op_1+12>:       xor    %edi,%eax
   0x40250e <p6_op_1+14>:       ret
   0x40250f:    nop
   0x402510 <p6_op_2>:  endbr64
   0x402514 <p6_op_2+4>:        mov    %edi,%eax
   0x402516 <p6_op_2+6>:        ror    $0x2,%eax
   0x402519 <p6_op_2+9>:        ret
   0x40251a:    nopw   0x0(%rax,%rax,1)
   0x402520 <p6_op_3>:  endbr64
   0x402524 <p6_op_3+4>:        mov    %edi,%eax
   0x402526 <p6_op_3+6>:        xor    $0xe816,%eax
   0x40252b <p6_op_3+11>:       lea    (%rax,%rdi,2),%eax
   0x40252e <p6_op_3+14>:       ret
   0x40252f:    nop
   0x402530 <p6_op_4>:  endbr64
   0x402534 <p6_op_4+4>:        mov    %edi,%eax
   0x402536 <p6_op_4+6>:        lea    0x0(,%rdi,8),%edx
   0x40253d <p6_op_4+13>:       shl    $0x5,%eax
   0x402540 <p6_op_4+16>:       sub    %edi,%edx
   0x402542 <p6_op_4+18>:       add    $0x4eee,%eax
   0x402547 <p6_op_4+23>:       xor    %edx,%eax
   0x402549 <p6_op_4+25>:       ret
```

```
   0x402648 <secret_phase+24>:  mov    %rdi,%rbx
(gdb) x/30i 0x4024f0
   0x4024f0 <p6_op_0>:  endbr64
   0x4024f4 <p6_op_0+4>:        mov    %edi,%eax
   0x4024f6 <p6_op_0+6>:        shr    $1,%eax
   0x4024f8 <p6_op_0+8>:        xor    %edi,%eax
   0x4024fa <p6_op_0+10>:       add    $0x203,%eax
   0x4024ff <p6_op_0+15>:       ret
```

```
0x402550 <p6_op_5>:  endbr64
0x402554 <p6_op_5+4>:        mov    %edi,%eax
0x402556 <p6_op_5+6>:        rol    $0x2,%eax
0x402559 <p6_op_5+9>:        xor    $0xc9d1,%eax
0x40255e <p6_op_5+14>:       ret
```

图 16　六个 p6_op 函数的完整反汇编结果

$$
\begin{aligned}
f_0(x) &= (x \oplus (x \gg 1)) + \texttt{0x203},\\
f_1(x) &= (x + \texttt{0x6fae}) \oplus (x \gg 1),\\
f_2(x) &= \mathrm{ROR}_{32}(x, 2),\\
f_3(x) &= (x \oplus \texttt{0xe816}) + 2x,\\
f_4(x) &= 7x \oplus (32x + \texttt{0x4eee}),\\
f_5(x) &= \mathrm{ROL}_{32}(x, 2) \oplus \texttt{0xc9d1}.
\end{aligned}
$$

其中，`ror` 和 `rol` 分别是循环右移和循环左移。与普通 `shr`、`shl` 不同，循环移出的位会从另一端重新进入。例如，两个循环移位函数在 C 中可分别表示为

$$
\begin{aligned}
\mathrm{ROR}_{32}(x, 2) &= (x \gg 2) \mid (x \ll 30),\\
\mathrm{ROL}_{32}(x, 2) &= (x \ll 2) \mid (x \gg 30).
\end{aligned}
$$

以 f_4 为例，`lea 0x0(,%rdi,8),%edx` 先得到 $8x$，`sub %edi,%edx` 将其改为 $7x$；与此同时，`eax` 左移 5 位后加上常数，最后两个中间值异或。这说明不能只按指令出现的顺序机械翻译，还需要分别追踪 `eax` 和 `edx` 的数据流。

两级分派与最终结果。 第一次间接调用返回后，程序先将结果与 `0x3c45` 异或，再把新值放入 `edi`，作为第二个动态选择函数的参数：

```
call *0x405e00(,%rax,8)
xor  $0x3c45,%eax
mov  %eax,%edi
call *0x405e00(%rbx)
add  $0x5125,%eax
```

由此可得完整计算关系

$$
\begin{aligned}
X_1 &= T[I_1](X_0),\\
X_2 &= T[I_2](X_1 \oplus \texttt{0x3c45}),\\
R &= X_2 + \texttt{0x5125} \pmod{2^{32}}.
\end{aligned}
$$

第二次调用的表基址虽然没有显式比例因子，但 `rbx` 本身已经是 8 的倍数，因此仍然访问同一张函数表。函数尾部的 `leave` 只负责恢复栈帧；其后的 `add` 仍属于返回值计算，不能因为看到 `leave` 就提前结束分析。

7.2 完整代码

```
static uint32_t p6_op_0(uint32_t x) {
    return (x ^ (x >> 1)) + 0x203;
}
static uint32_t p6_op_1(uint32_t x) {
    return (x + 0x6fae) ^ (x >> 1);
}
static uint32_t p6_op_2(uint32_t x) {
    return (x >> 2) | (x << 30);
}
static uint32_t p6_op_3(uint32_t x) {
    return (x ^ 0xe816) + 2 * x;
}
static uint32_t p6_op_4(uint32_t x) {
    return (7 * x) ^ (32 * x + 0x4eee);
}
static uint32_t p6_op_5(uint32_t x) {
    return ((x << 2) | (x >> 30)) ^ 0xc9d1;
}
typedef uint32_t (*op_func)(uint32_t);
static op_func ops[8] = {
    p6_op_1, p6_op_1, p6_op_4, p6_op_3,
    p6_op_5, p6_op_1, p6_op_2, p6_op_0
};

int solve_phase_6(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 1;
    uint32_t x = (challenge >> 0x20) ^ (challenge >> 0x6);
    uint32_t idx1 = challenge & 0x7;
    uint32_t idx2 = (challenge & 0x38) >> 3;
    x = ops[idx1](x);
    x ^= 0x3c45;
    x = ops[idx2](x);
    x += 0x5125;
    values[0] = x;
    return write_answer(out, cap, values, count);
}
```

7.3 运行结果



图 17　Phase 6 公开列表外种子运行结果

```
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
```

图 18　Phase 6 多种子测试结果

8 Secret 关：字符串门控与三字段校验

8.1 关键推导

Secret 关的求解函数除接收 challenge 外，还接收 64 位 context、输出缓冲区 out 及其容量 cap。secret_gate 先在栈上建立缓冲区，并使用 secret_copy 复制整行输入。缓冲区起点与关键检查地址分别为

```
lea   -0xd0(%rbp),%r12
...
cmpl  $0x4e45504f,-0xb0(%rbp)
jne   failure
lea   -0xac(%rbp),%rdi
call  secret_phase
```

缓冲区起点为 rbp-0xd0，而比较地址 rbp-0xb0 与其相差 0x20=32 字节。x86-64 使用小端序，常数 0x4e45504f 在内存中的字节序列为 4f 50 45 4e，即字符串 OPEN。随后传给 secret_phase 的地址 rbp-0xac 又比比较地址大 4，因此完整输入必须具有如下布局：

[32 个填充字符]OPEN[三个无符号整数]

程序还通过以下无符号比较限制整行长度：

```
call  strlen
sub   $0x24,%rax
cmp   $0x7f,%rax
jbe   valid_length
```

该判断等价于

$$36 \le \mathrm{strlen}(input) \le 163. \tag{19}$$

长度检查通过后，secret_copy 逐字节复制输入，并在读到 '\0' 后结束：

```
movzbl (%rsi,%rax,1),%edx
mov    %dl,(%rdi,%rax,1)
add    $0x1,%rax
test   %dl,%dl
jne    copy_loop
```

因此终止字符也会被写入目标缓冲区。最大长度 163 与局部缓冲区的 164 字节容量相匹配，恰好为末尾的 '\0' 保留一个字节。

三个字段的解析。 secret_phase 的三个参数依次为数字字段字符串、challenge 和 context。函数先调用 valid_u32_fields(input,3)，随后调用 sscanf：

```
lea  -0x24(%rbp),%r8
```

```
lea  -0x28(%rbp),%rcx
mov  $0x403183,%esi
lea  -0x2c(%rbp),%rdx
mov  %rbx,%rdi
call __isoc99_sscanf
cmp  $0x3,%eax
jne  failure
```

使用 GDB 命令 `x/s 0x403183` 得到格式字符串 `"%u %u %u"`，所以三个字段均为以空格分隔的十进制 `uint32_t`。记它们依次为 v_0、v_1 和 v_2。

secret_mix 与公共中间量。 辅助函数 `secret_mix` 的关键汇编为

```
mov  %edi,%eax
xor  $0x7479,%edi
shr  $0x3,%eax
xor  $0x916f,%eax
add  %edi,%eax
ret
```

因此，对任意 32 位输入 k，其返回值为

$$\mathrm{mix}(k) = (k \oplus \texttt{0x7479}) + ((k \gg 3) \oplus \texttt{0x916f}) \pmod{2^{32}}. \tag{20}$$

与求解代码中的变量一致，定义

$$x = \mathrm{low}_{32}(\texttt{challenge}) \oplus \mathrm{high}_{32}(\texttt{challenge}),$$
$$y = \mathrm{low}_{32}(\texttt{context}) \oplus \mathrm{high}_{32}(\texttt{context}).$$

汇编先分别将 `challenge` 和 `context` 右移 32 位，再与各自的低 32 位异或，因而 `x` 和 `y` 均是对应 64 位输入的高低半部折叠值；传入 `secret_mix` 的参数为 `x ^ y`。

三个目标字段的推导。 第一个字段由

```
mov  %r13d,%ecx
and  $0x3,%ecx
cmp  %ecx,-0x2c(%rbp)
```

直接得到代码中的第一个输出字段

$$\texttt{out_1} = \texttt{context} \,\&\, 3. \tag{21}$$

随后程序计算

```
imul  $0x7479,%ecx,%r9d
shr   $0x10,%r13
```

```
xor   %r13d,%r9d
...
call  secret_mix
xor   %r9d,%eax
cmp   %eax,-0x28(%rbp)
```

其中 ecx 保存 out_1，故代码中的 z 及第二个输出字段为

$$\begin{aligned} \mathsf{z} &= (\mathsf{out_1} \times \mathtt{0x7479}) \oplus \mathrm{low}_{32}(\mathsf{context} \gg 16), \\ \mathsf{out_2} &= \mathrm{mix}(\mathsf{x} \oplus \mathsf{y}) \oplus \mathsf{z}. \end{aligned}$$

验证第二个字段后，eax 中仍为 out_2。最后一段汇编为

```
shr  $0x3,%esi
lea  0x916f(%r8,%rcx,1),%edx
add  %esi,%edx
xor  %edx,%eax
cmp  -0x24(%rbp),%eax
```

此时 esi 原先保存 x，r8d 保存 y，ecx 保存 out_1，因此第三个字段为

$$\mathsf{out_3} = \mathsf{out_2} \oplus (\mathtt{0x916f} + \mathsf{y} + \mathsf{out_1} + (\mathsf{x} \gg 3)) \pmod{2^{32}}. \tag{22}$$

8.2 完整代码

```c
static uint32_t secret_mix(uint32_t k) {
    return (k ^ 0x7479) + ((k >> 0x3) ^ 0x916f);
}

int solve_secret(uint64_t challenge, uint64_t context,
                 char *out, size_t cap) {
    uint32_t x = challenge ^ (challenge >> 0x20);
    uint32_t y = context ^ (context >> 0x20);
    uint32_t z = ((context & 0x3) * 0x7479)
                 ^ (context >> 0x10);
    uint32_t out_1 = context & 0x3;
    uint32_t x_1 = secret_mix(x ^ y);
    uint32_t out_2 = x_1 ^ z;
    uint32_t out_3 = out_2
                     ^ (0x916fu + y + out_1 + (x >> 3));
    int n = snprintf(
        out, cap,
        "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN%u %u %u",
        out_1, out_2, out_3
    );
    if (n < 0 || (size_t)n >= cap)
        return 1;
    return 0;
}
```

8.3 运行结果

使用公开列表外的 seed 1234 进行单次运行，Secret 关成功解除；随后对公开种子集合进行批量测试，各组结果均为 PASS。

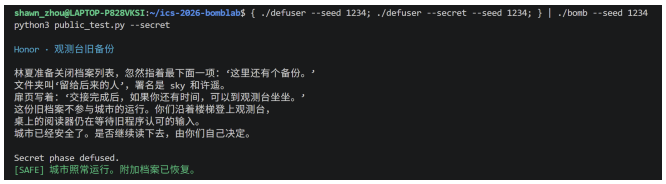


图 19　Secret 关公开列表外种子运行结果

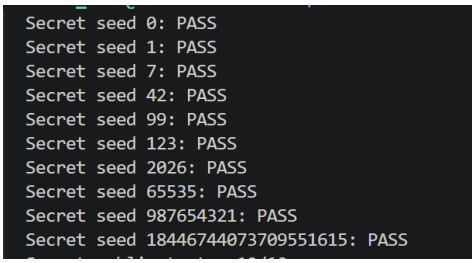


图 20　Secret 关多种子测试结果

9 参考资料与辅助工具说明

本实验的调用约定、寄存器宽度和指令语义以课程材料、实验 README 及程序反汇编为主要依据。使用 `objdump` 提取各关关键函数的反汇编，使用 GDB 查看跳转表、链表节点、函数指针表和格式字符串；使用编译器构建求解器，并通过 `tests/public.json` 中的公开种子、公开列表外种子的单次运行以及批量测试脚本核验结果。

AI 辅助使用情况。　本实验使用 Codex 作为辅助工具，具体用于以下方面：

1. 解释 AT&T 汇编语法、调用约定及部分指令语义；
2. 核对寄存器追踪、计算关系和边界条件是否与关键汇编一致；
3. 协助检查 C 语言实现与推导公式的一致性，并定位编译或运行中的问题；
4. 协助整理报告结构、文字表述和 LaTeX 排版。

AI 仅用于解释、检查和整理，没有替代实验的核心分析与验证工作。各关关键指令的筛选、计算关系的恢复、求解代码的实现以及测试结果的确认主要由作者独立完成；所有 AI 建议均结合原始反汇编和实际运行结果进行复核，不以 AI 输出本身作为实验结论。